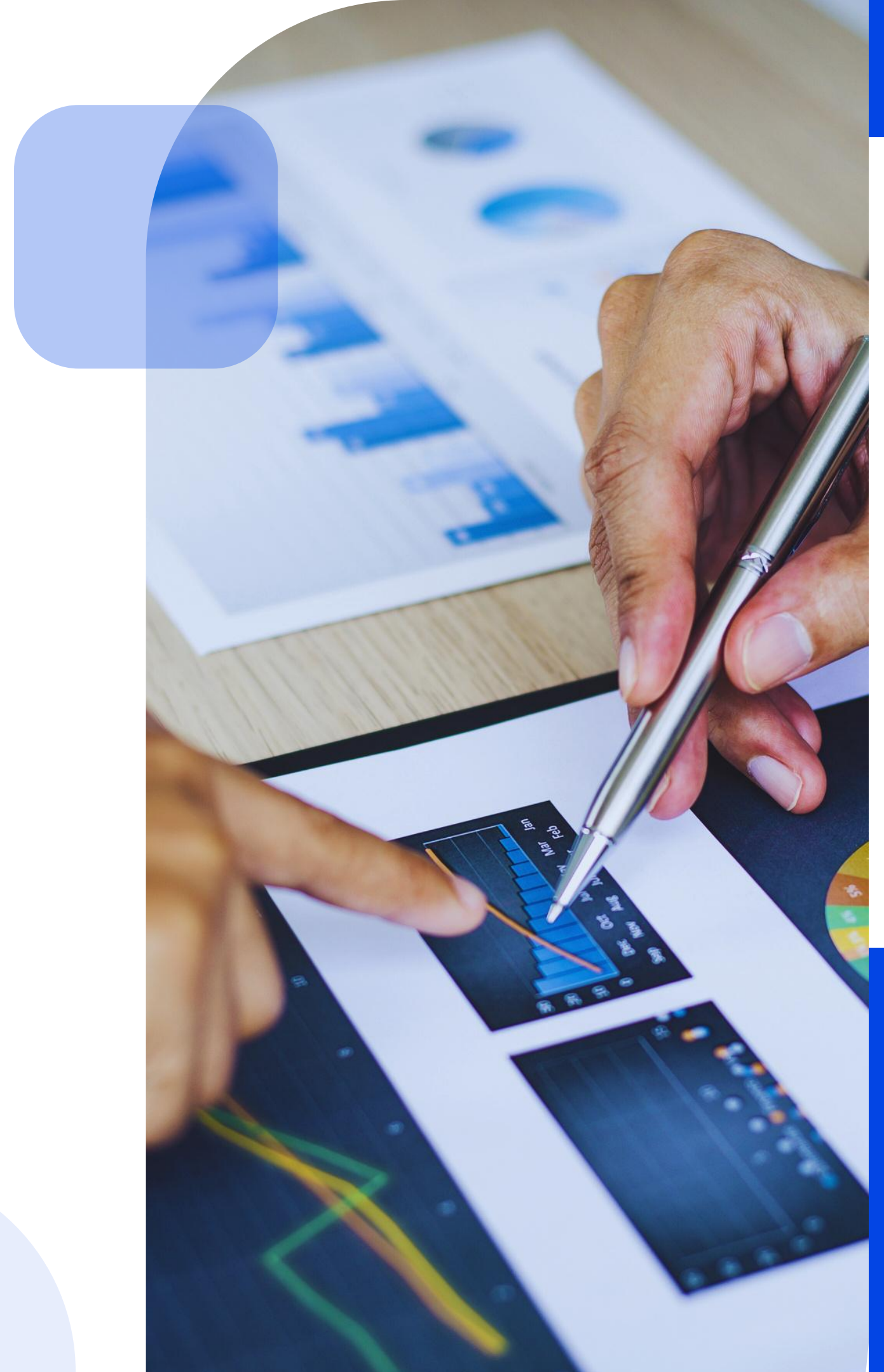


# COMMODITY DATA ANALYSIS

*Exploratory Data Analysis Using MySQL and Python*



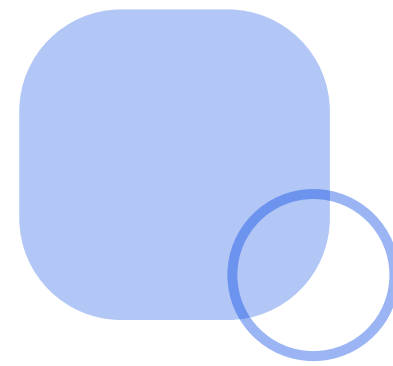
# OVERVIEW

---

This project focuses on analyzing regional market trends, pricing architecture using an extensive commodity dataset across India from 2019 to 2021.

Data from three tables was cleaned using MySQL and analyzed using Python;  
Through Univariate, Bivariate, and Multivariate EDA.

The project isolates high-cost logistics traps, validates the baseline price stability of everyday food staples, and identifies high-margin growth opportunities by premium performance drivers.



## Analysis Approach

01

### **Univariate Analysis**

Analyzed individual variables to understand distributions, frequencies, and outliers.

02

### **Bivariate Analysis**

Explored relationships between two variables to identify trends and patterns.

03

### **Multivariate Analysis**

Analyzed interactions among multiple variables to uncover deeper business insights.

## Dataset Overview

The project uses three datasets:

1. **price\_details** – Contains commodity IDS, region IDs, dates, and retail prices
2. **commodities\_info** – Contains commodity IDs, names, varieties, and categories
3. **region\_info** – Contains region IDs, state and city information

Data integrated using **SQL joins** to create a unified analytical dataset.

## Data Cleaning

- Checked for null values
- Checked duplicate records
- Verified data types
- Identified blank spaces in the **Variety column**(commodities\_info)
- Replaced blank variety values with 'Unknown'
- Performed data validation and quality checks using MySQL.

## Tools & Technology Used

- **MySQL** – Data Cleaning, Validation, and Table Joins
- **Python** – Data Analysis
- **Pandas** – Data Manipulation and Transformation
- **Matplotlib** – Data Visualization
- **Seaborn** – Statistical Visualization
- **Jupyter Notebook** – Analysis and Reporting Environment

## Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlalchemy
```

## Step 2: Load Dataset from mysql

```
engine = sqlalchemy.create_engine('mysql+pymysql://root:123456789@localhost:3306/commodity_db')
```

## Step 3: Join tables using sql query

### Join tables to One Dataframe

```
[6]: query = """ Select c.Commodity,c.Variety,c.Unit,c.Category,r.Centre,r.State,p.Date,p.Retail_Price
from price_details p
join commodities_info c
on p.Commodity_id=c.Id
join region_info r
on p.Region_Id=r.Id"""
```

## Step 4: Converted Query into DataFrame

```
[7]: df = pd.read_sql(query, engine)
df
```

```
[7]:
```

|       | Commodity        | Variety         | Unit     | Category | Centre         | State          | Date       | Retail_Price |
|-------|------------------|-----------------|----------|----------|----------------|----------------|------------|--------------|
| 0     | Moong            | Split           | Kg.      | Food     | Chittoor       | Andhra Pradesh | 2019-01-01 | 90.0         |
| 1     | Moong            | Split           | Kg.      | Food     | Guntur         | Andhra Pradesh | 2019-01-01 | 84.0         |
| 2     | Moong            | Split           | Kg.      | Food     | Kurnool        | Andhra Pradesh | 2019-01-01 | 85.0         |
| 3     | Moong            | Split           | Kg.      | Food     | Vishakhapatnam | Andhra Pradesh | 2019-01-01 | 90.0         |
| 4     | Moong            | Split           | Kg.      | Food     | Dibrugarh      | Assam          | 2019-01-01 | 77.0         |
| ...   | ...              | ...             | ...      | ...      | ...            | ...            | ...        | ...          |
| 90341 | Cycle Tyre       | UNKNOWN         | Each     | Non Food | Patna          | Bihar          | 2020-12-01 | 387.0        |
| 90342 | Detergent Powder | Nirma/Any Other | Per Kg.  | Non Food | Patna          | Bihar          | 2020-12-01 | 53.0         |
| 90343 | Diesel           | UNKNOWN         | Litre    | Non Food | Patna          | Bihar          | 2020-12-01 | 60.0         |
| 90344 | Electric bulb    | UNKNOWN         | 40 Watts | Non Food | Patna          | Bihar          | 2020-12-01 | 18.0         |
| 90345 | Exercise Book    | UNKNOWN         | Each     | Non Food | Patna          | Bihar          | 2020-12-01 | 48.0         |
| 90346 |                  |                 |          |          |                |                |            |              |

90346 rows x 8 columns



# Univariate Analysis

## Numerical Variable: Retail Price

Retail\_Price is our primary numerical continuous variable. To check its distribution shape (is it skewed?) and see if there are extreme outliers.

```
print("\n--- Price Summary Statistics ---")
print(df["Retail_Price"].describe())
```

```
--- Price Summary Statistics ---
count      90346.000000
mean       128.944969
std        201.574124
min         1.000000
25%        30.000000
50%        54.000000
75%       130.000000
max       2750.000000
Name: Retail_Price, dtype: float64
```

This shows **mathematical summary** of our Retail\_Price column, giving us an instant overview of our data's range, average, and middle ground without making you look through all 90,000+ rows.

What is the overall distribution shape of our commodity retail prices & Do we have unusual or extreme pricing anomalies in our market data, and what is the typical boundary for a 'normal' commodity price?

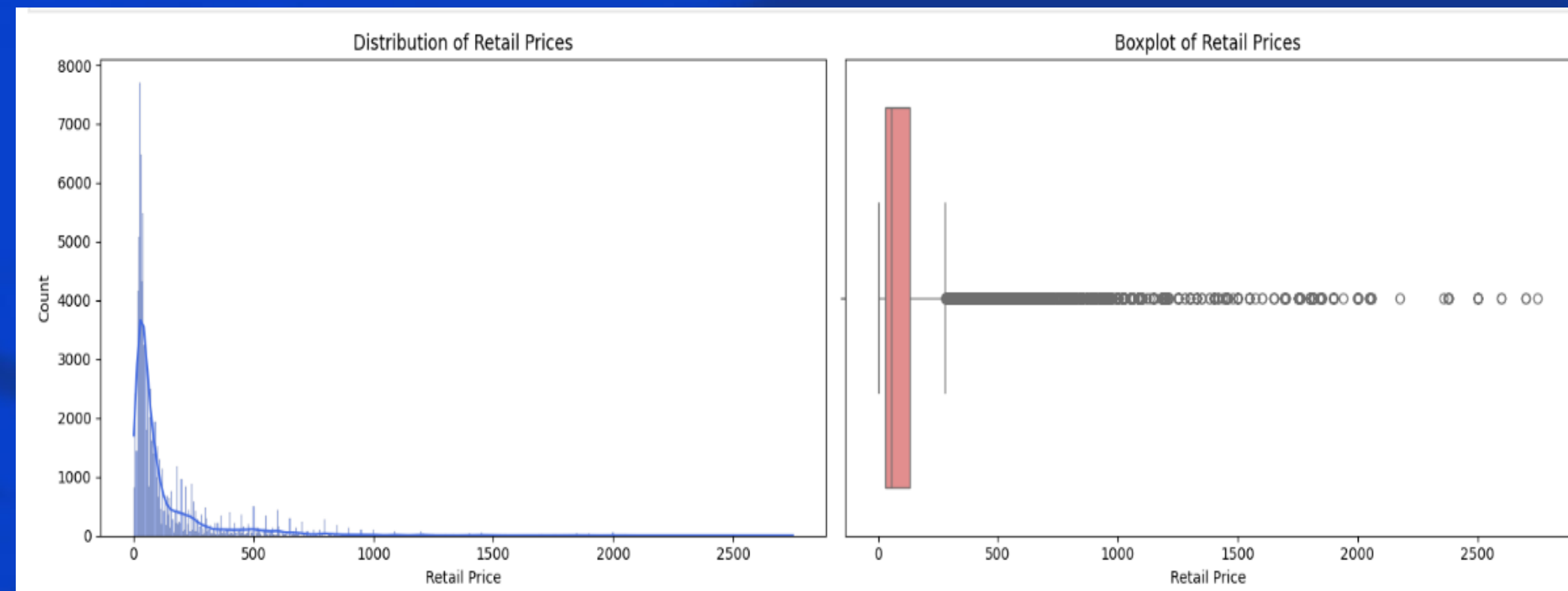
```
# Plotting Distribution and Outliers side-by-side
#-----

fig, axes = plt.subplots(1, 2, figsize=(16,5))

# 1. Histogram / KDE -----
sns.histplot(df["Retail_Price"], kde=True, ax=axes[0], color="royalblue")
axes[0].set_title("Distribution of Retail Prices")
axes[0].set_xlabel("Retail Price")

# 2. Boxplot (Great for spotting outliers) -----
sns.boxplot(x=df["Retail_Price"], ax=axes[1], color="lightcoral")
axes[1].set_title("Boxplot of Retail Prices")
axes[1].set_xlabel("Retail Price")

plt.tight_layout()
plt.show()
```



### Business Insights:

**A High-Value "Long Tail":** The long tail stretching out past 2,500 represents premium, bulk, or industrial non-food goods. They have a completely different price scale and account for low-volume, high-value revenue.

**Outliers to Clean:** The long trail of circles in the boxplot highlights extreme data points.

## Categorical Variables (Top 10 Analysis)

Columns like Commodity, Variety, Unit, Category, Centre, and State are categorical. Looking at the raw counts for columns with dozens of entries (like Centre or Commodity) will clutter the screen, so focus on the Top 10 most frequent records for each.

```
categorical_columns = ["Category", "Unit", "State", "Commodity", "Centre"]

for col in categorical_columns:
    print(f"\n--- Top 10 Most Frequent Categories in '{col}' ---")
    print(df[col].value_counts().head(10))
```

```
--- Top 10 Most Frequent Categories in 'Category' ---
Category
Food          77164
Non Food      13182
Name: count, dtype: int64

--- Top 10 Most Frequent Categories in 'Unit' ---
Unit
Kg.           51588
Kg/Litre      6213
100gm. Pkt    5691
Litre         3504
Dozen         2971
Each          1991
Metre         1649
Peice         1469
400/800 Gm    1453
80gm. Pkt     1406
Name: count, dtype: int64

--- Top 10 Most Frequent Categories in 'State' ---
State
Uttar Pradesh    8631
Madhya Pradesh   7939
Rajasthan        7492
Tamil Nadu       6639
Andhra Pradesh   5991
Assam            5726
Kerala           5678
Maharashtra      5620
West Bengal      4478
Karnataka        3695
Name: count, dtype: int64

--- Top 10 Most Frequent Categories in 'Commodity' ---
Commodity
Rice           4577
Gram           2969
Milk           2887
Salt           2828
Coconut        2730
Fish           2680
Tea            2662
Wheat          2455
Coffee         1873
Potato         1567
Name: count, dtype: int64
```

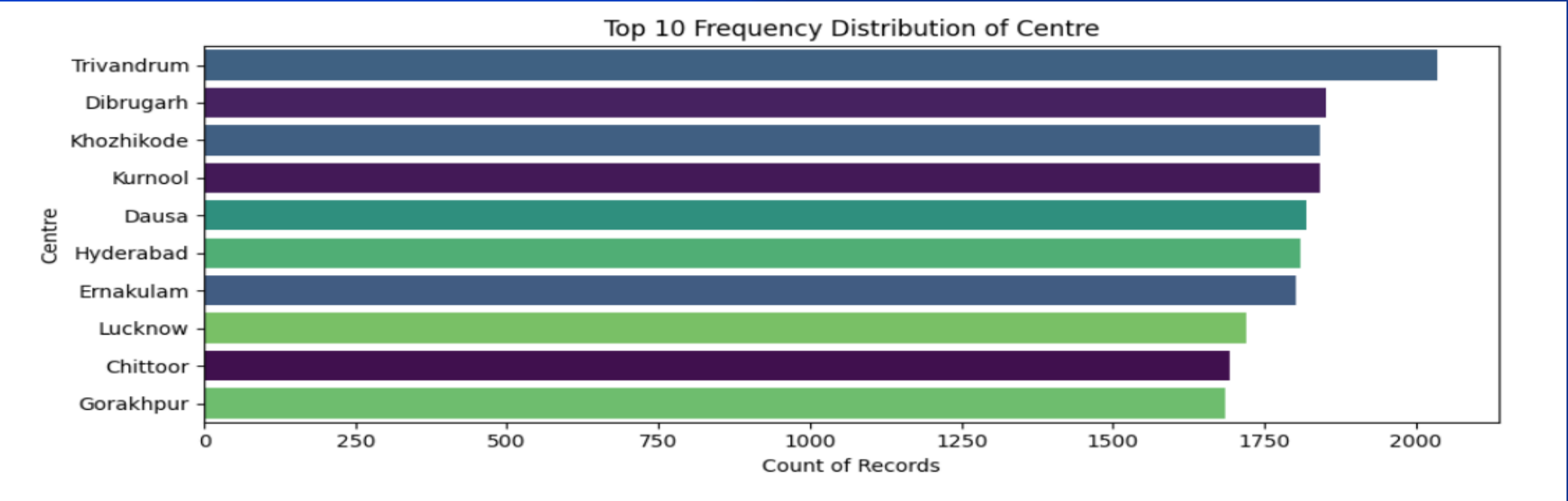
## Top 10 Frequency Distribution of Centre(City)

```
# Plotting -----

plt.figure(figsize=(10, 4))
top_10_order = df[col].value_counts().head(10).index

sns.countplot(
    data=df,
    y=col,
    order=top_10_order,
    palette="viridis",
    hue=col,
    legend=False,
)

plt.title(f"Top 10 Frequency Distribution of {col}")
plt.xlabel("Count of Records")
plt.ylabel(col)
plt.tight_layout()
plt.show()
```



### Business Insights:

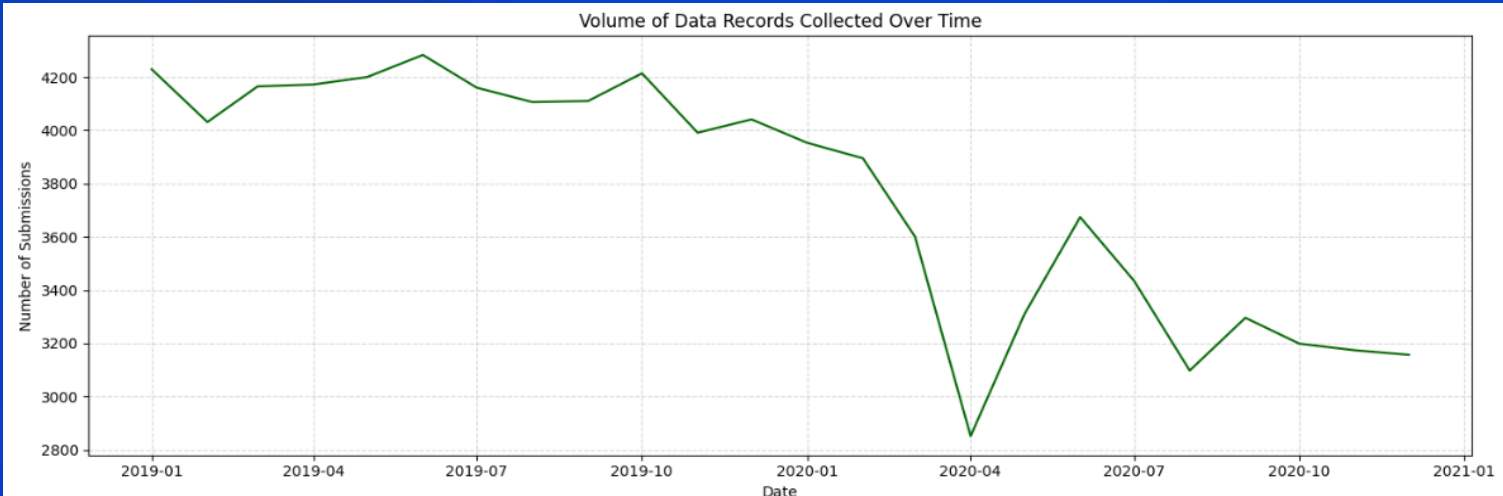
- Balanced Regional Reporting:** The data collection is incredibly uniform. Except for a minor lead by Trivandrum (over 2,000 records), all other top centres like Dibrugarh, Khozhikode, and Kurnool hover neck-and-neck around the **1,700 to 1,850 records** mark.
- Operational Focus:** These 10 cities represent most active, data-rich supply hubs. When testing regional predictive pricing models or launching logistics updates, these specific centres are your baseline "gold standard" locations due to their consistent logging consistency.

## Date Analysis (Date Frequency Over Time)

For a standalone Date column, Univariate analysis helps to see if the data collection was consistent month-to-month, or if there are massive gaps where data wasn't recorded.

```
date_distribution = df["Date"].value_counts().sort_index()

plt.figure(figsize=(14, 5))
plt.plot(
    date_distribution.index,
    date_distribution.values,
    color="darkgreen",
    linewidth=1.5,
)
plt.title("Volume of Data Records Collected Over Time")
plt.xlabel("Date")
plt.ylabel("Number of Submissions")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



### Business insights:

**What the chart shows:** Our data collection was highly stable throughout 2019 (averaging over 4,100 records monthly). However, there is a **massive, sharp crash in April 2020**, where submissions plummeted to an all-time low of roughly 2,850.

**The Reality Behind It:** This perfectly captures the operational disruption caused by the strict, nationwide COVID-19 lockdowns in India starting in late March 2020. Market data collectors physically could not visit retail centers to log prices, causing a severe drop in data reporting.

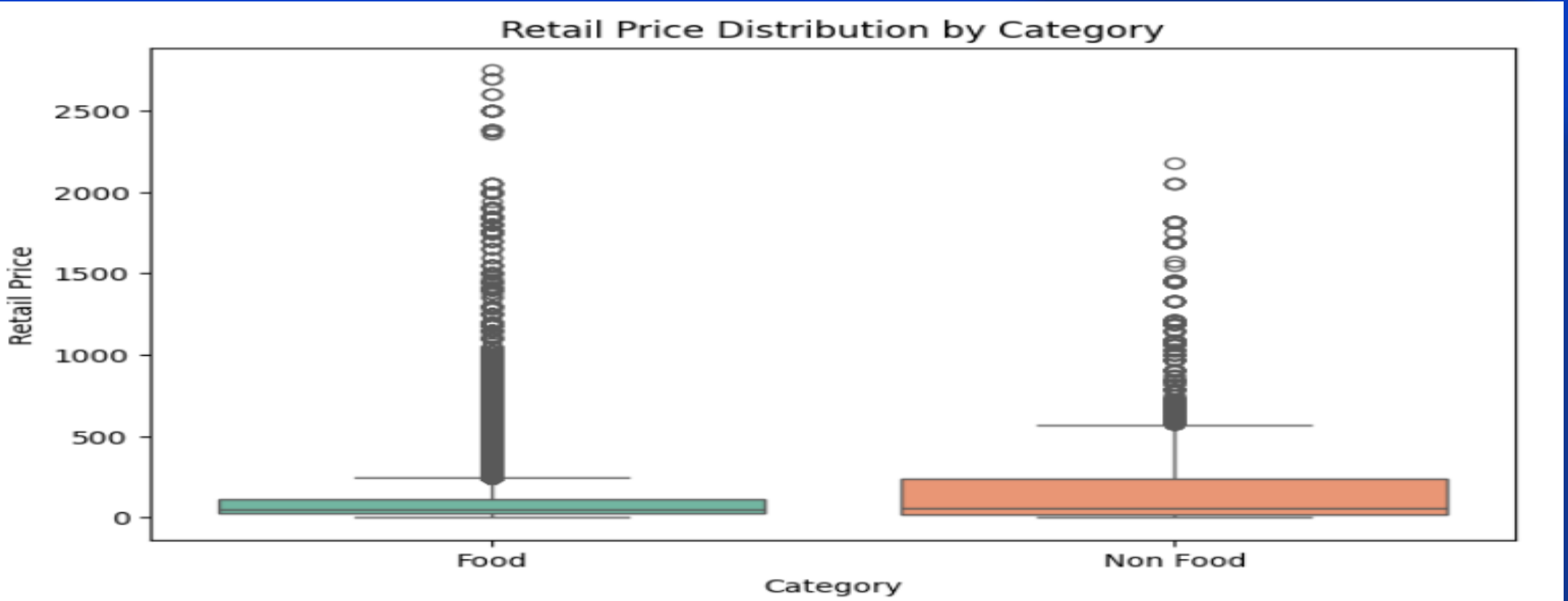
## Bivariate Analysis

### Categorical vs Numerical: Price by Category & State

**Question 1: Do non-food items have a significantly higher or more volatile price range compared to food items?**

```
# 1. Price Distribution by Category (Food vs Non-Food)

plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="Category", y="Retail_Price", palette="Set2", hue="Category", legend=False)
plt.title("Retail Price Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Retail Price")
plt.show()
```



### Business Insights:

Based on the boxplot comparing **Food** vs. **Non-Food** prices, here are the direct business insights:

- Non-Food Items Have Higher Core Prices:** The orange box (Non-Food) sits noticeably higher than the green box (Food). This tells us that the middle 50% of our non-food inventory naturally commands a higher baseline price than food staples.
- Food Category Contains More Extreme Spikes:** Look at the trail of outlier circles. Even though food is cheaper on average, it contains the absolute highest extreme pricing anomalies in the entire dataset

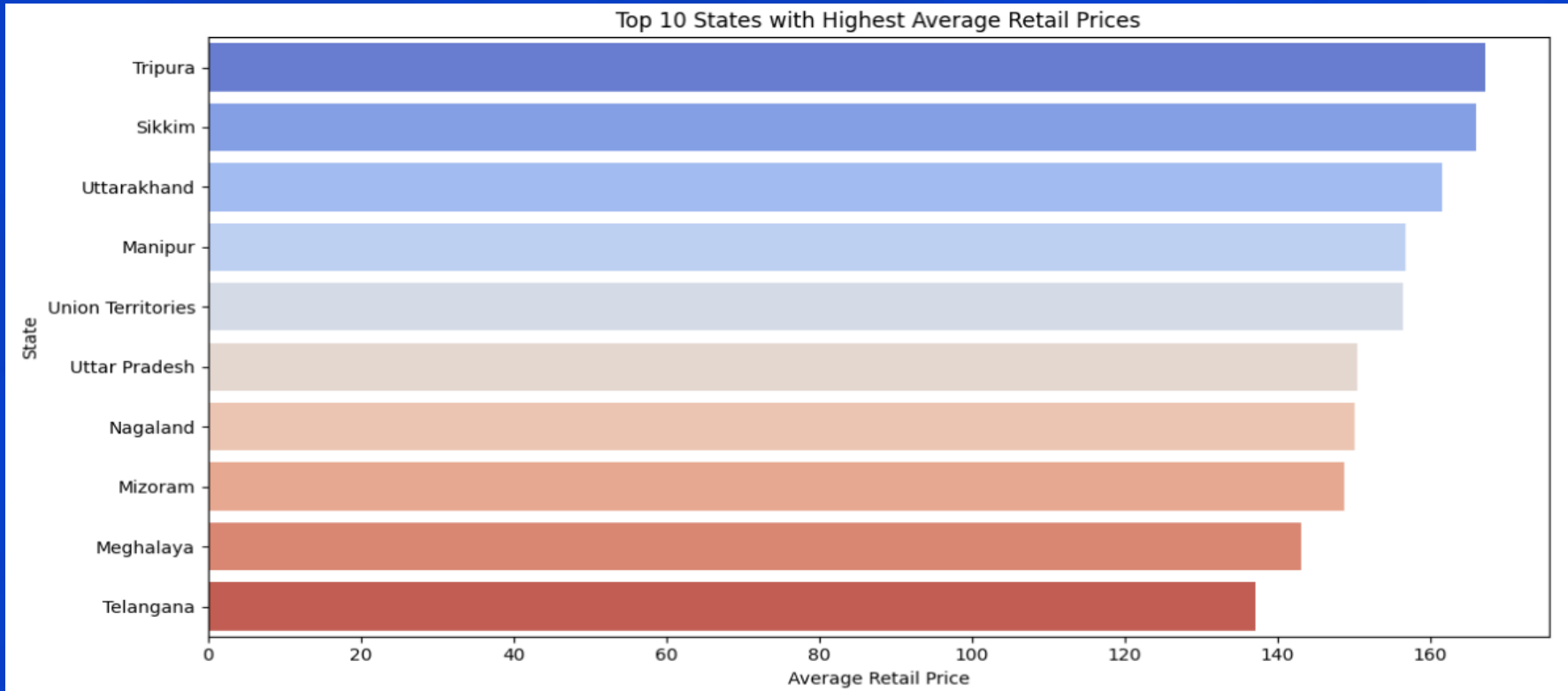


Question 2: Which states have the highest average cost for commodities, and which are the cheapest?

```
# 2. Average Price by State (Top 10 Most Expensive States on average)

state_price = df.groupby("State")["Retail_Price"].mean().sort_values(ascending=False).head(10)

plt.figure(figsize=(12, 6))
sns.barplot(x=state_price.values, y=state_price.index, palette="coolwarm", hue=state_price.index, legend=False)
plt.title("Top 10 States with Highest Average Retail Prices")
plt.xlabel("Average Retail Price")
plt.ylabel("State")
plt.tight_layout()
plt.show()
```



**Business Insights:**

**Geographic Premium in the Northeast:** The chart is heavily dominated by Northeastern states (Tripura, Sikkim, Manipur, Nagaland, Mizoram, and Meghalaya). **Tripura** and **Sikkim** lead the country as the absolute most expensive regions on average.

**The Logistical Inflation Trap:** These specific states share a common geographic trait—they are largely mountainous, remote, or landlocked regions.

Time-Series Bivariate Analysis: Price Over Time

Question 3: How do retail prices fluctuate over time, and do food and non-food items follow the same trend?

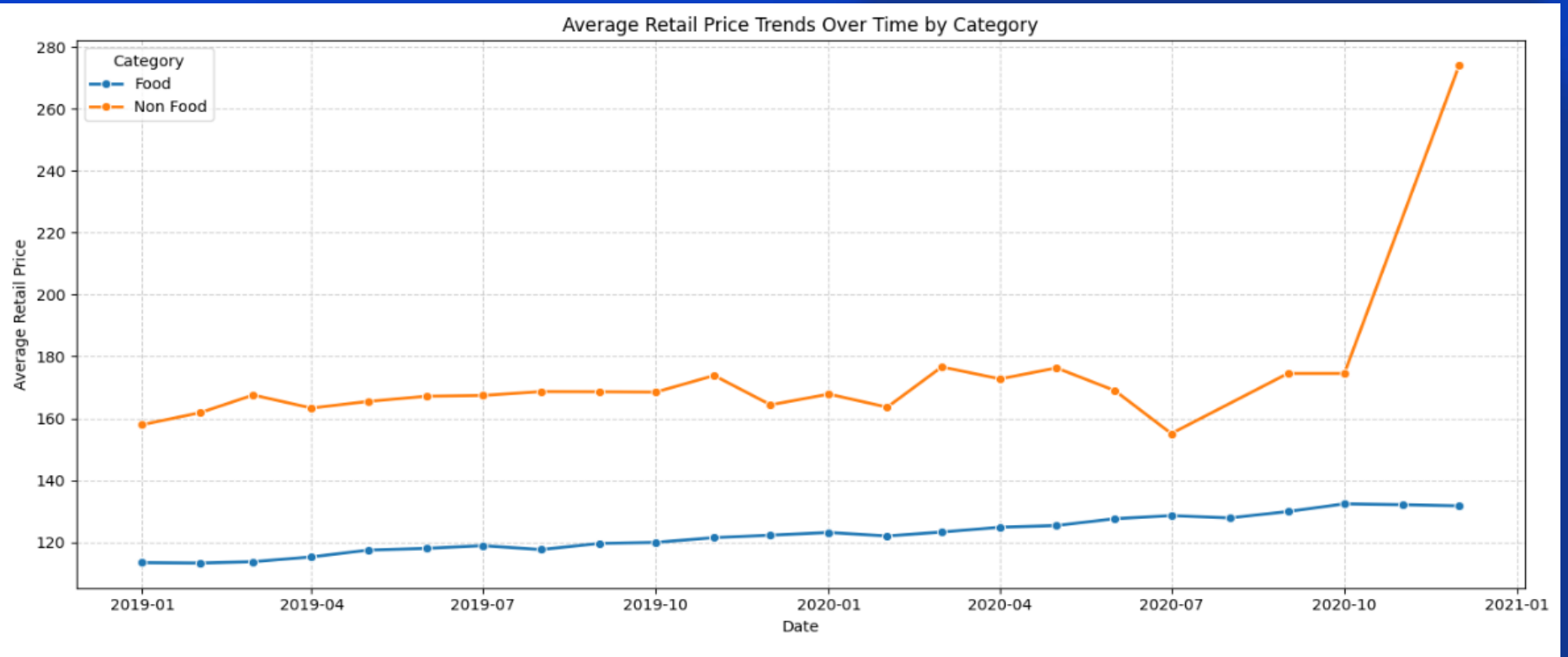
```
# Ensure Date is in datetime format-----

df["Date"] = pd.to_datetime(df["Date"])

# Group by Date and Category to get the mean price over time-----

time_trend = df.groupby(["Date", "Category"])["Retail_Price"].mean().reset_index()

# Plotting the trends-----
plt.figure(figsize=(14, 6))
sns.lineplot(data=time_trend, x="Date", y="Retail_Price", hue="Category", marker="o", linewidth=2)
plt.title("Average Retail Price Trends Over Time by Category")
plt.xlabel("Date")
plt.ylabel("Average Retail Price")
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()
```



**Business Insights:**

**The Non-Food Inflation Shock:** While **Food** prices (the blue line) remained incredibly stable, smooth, and predictable throughout 2019 and 2020, **Non-Food** prices (the orange line) experienced a massive, unprecedented pricing spike at the end of 2020, skyrocketing from roughly **175 to over 270** in just two months.

**Economic Resiliency vs. Volatility:** Daily food essentials are highly resilient to dramatic market swings, likely due to government price controls or steady local demand. Non-food items, however, are highly vulnerable to macroeconomic market disruptions.



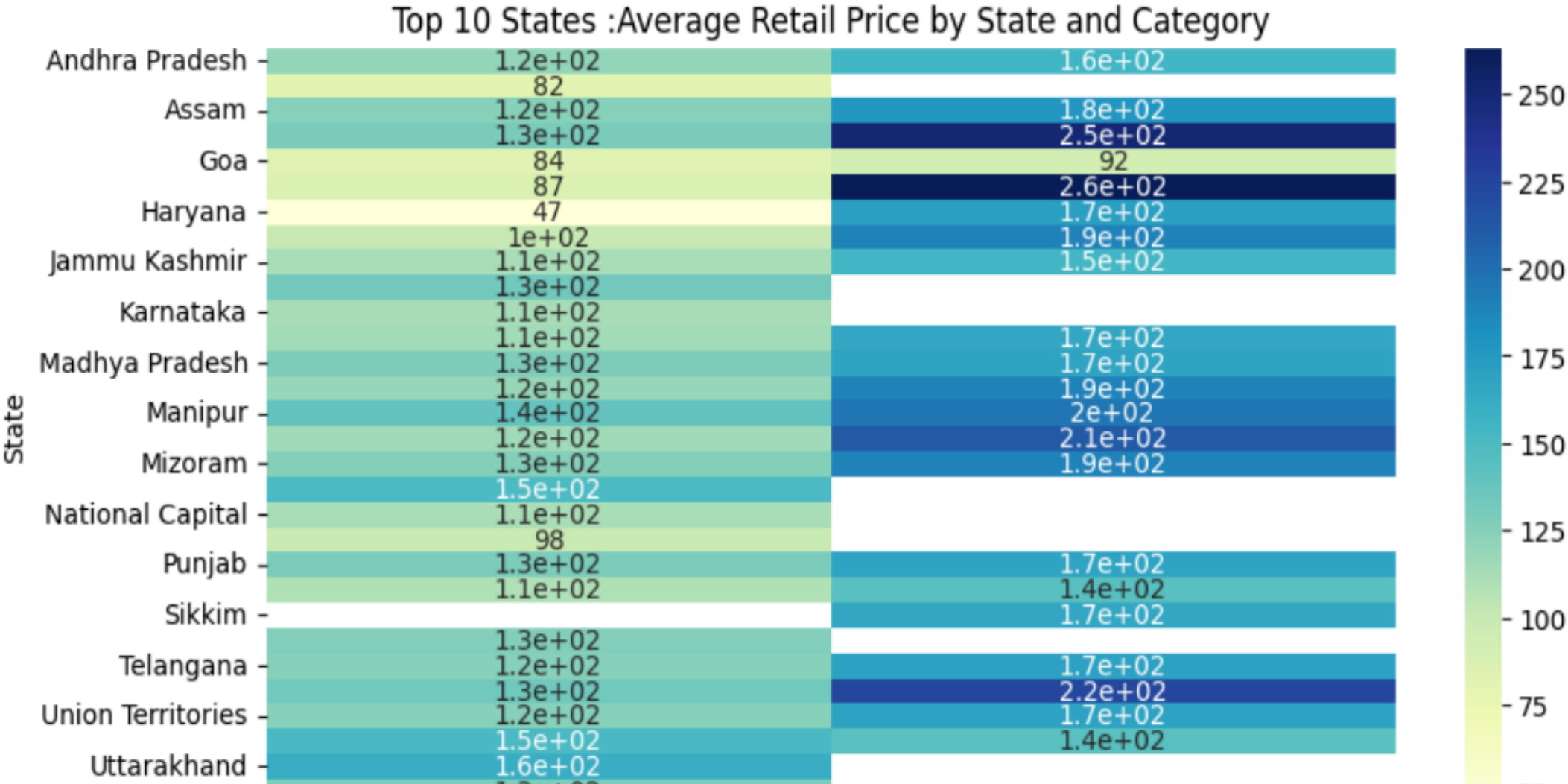
# Multivariate Analysis

```
# Question
# Which states have the highest average prices across different categories?
```

```
pivot1 = df.pivot_table(
    values='Retail_Price',
    index='State',
    columns='Category',
    aggfunc='mean'
)
pivot1.head()
```

```
plt.figure(figsize=(10,5))
sns.heatmap(pivot1, annot=True, cmap='YlGnBu')
plt.title('Top 10 States :Average Retail Price by State and Category')
plt.show()
```

| Category          | State      |            |
|-------------------|------------|------------|
|                   | Food       | Non Food   |
| Andhra Pradesh    | 120.585198 | 155.332029 |
| Arunachal Pradesh | 81.549550  | NaN        |
| Assam             | 124.875329 | 177.863167 |
| Bihar             | 129.663437 | 250.610390 |
| Goa               | 83.561404  | 92.297665  |



Business Insight:

This analysis helps identify states with higher average commodity prices across different categories. It highlights regional price variations and supports pricing, supply chain, and market planning decisions.

```
# Question
# Which commodities are expensive in different states?
```

```
top_commodities = df['Commodity'].value_counts().head(5).index
top_states = df['State'].value_counts().head(5).index

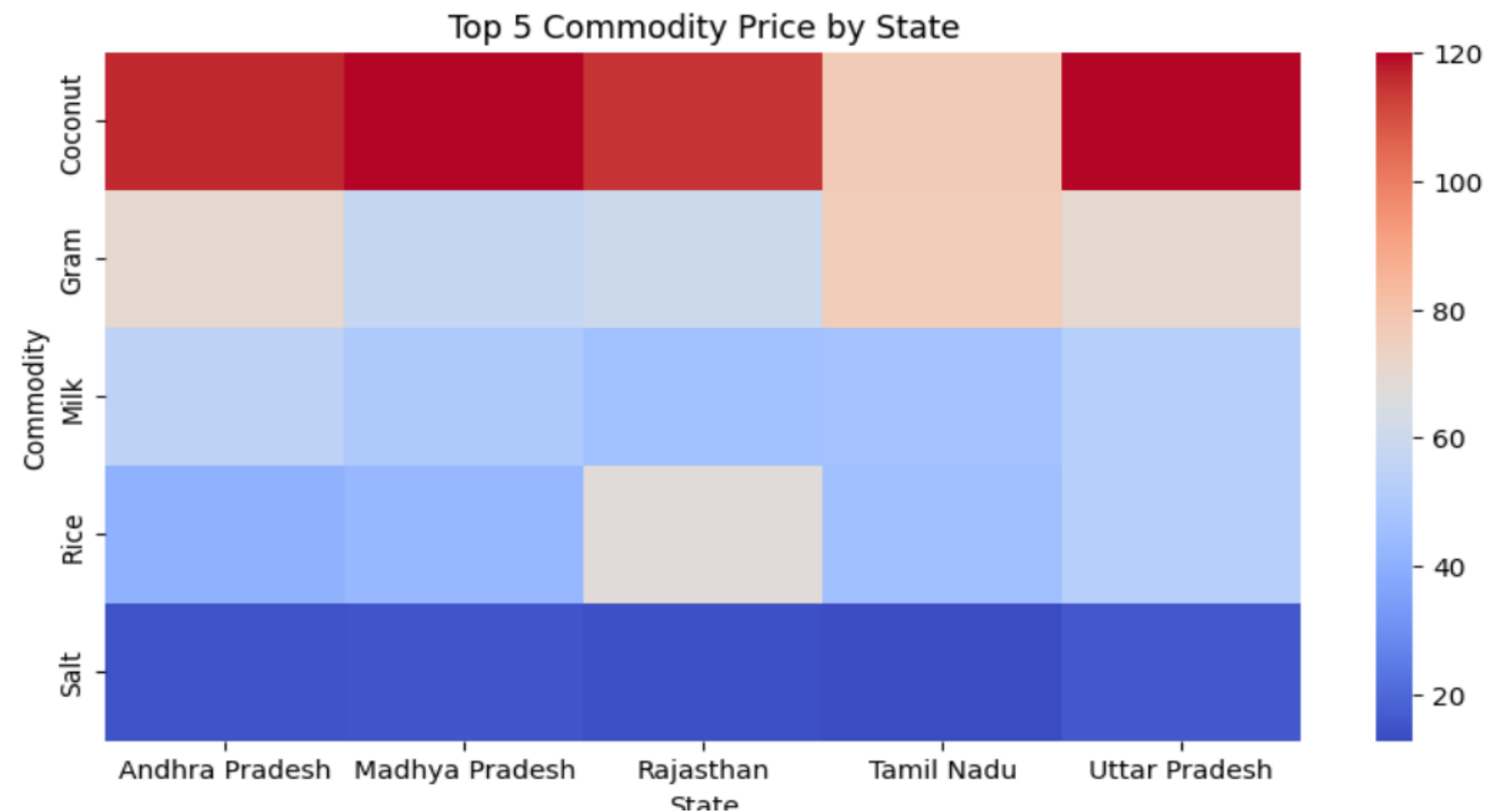
filtered_df = df[
    (df['Commodity'].isin(top_commodities)) &
    (df['State'].isin(top_states))
]

pivot2 = filtered_df.pivot_table(
    values='Retail_Price',
    index='Commodity',
    columns='State',
    aggfunc='mean'
)
print("\n-----")
print(pivot2)
```

```
plt.figure(figsize=(10,5))
sns.heatmap(pivot2, cmap='coolwarm')
plt.title('Top 5 Commodity Price by State')
plt.show()
```

| State     | Andhra Pradesh | Madhya Pradesh | Rajasthan  | Tamil Nadu |
|-----------|----------------|----------------|------------|------------|
| Commodity |                |                |            |            |
| Coconut   | 116.544872     | 120.013889     | 114.920502 | 77.024510  |
| Gram      | 70.378205      | 57.263889      | 59.795833  | 76.133333  |
| Milk      | 55.051282      | 49.937500      | 46.204167  | 46.626087  |
| Rice      | 40.632479      | 42.951389      | 67.702778  | 45.291429  |
| Salt      | 14.583333      | 15.104167      | 14.012500  | 12.870833  |

| State     | Uttar Pradesh |
|-----------|---------------|
| Commodity |               |
| Coconut   | 119.635417    |
| Gram      | 70.451389     |
| Milk      | 52.239583     |
| Rice      | 52.312500     |
| Salt      | 15.940972     |



Business Insight:

This analysis identifies commodities that are relatively expensive in specific states.

It helps understand regional demand-supply differences and supports sourcing and inventory decisions.

```
# Question
```

```
# How do food and non-food prices change over time?
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
trend = df.groupby([  
    pd.Grouper(key='Date', freq='ME'),  
    'Category'  
])['Retail_Price'].mean().reset_index()
```

```
print(trend.head())
```

```
plt.figure(figsize=(10,5))
```

```
for category in trend['Category'].unique():  
    data = trend[trend['Category'] == category]  
    plt.plot(data['Date'], data['Retail_Price'], label=category)
```

```
plt.title('Average Retail Price Trend by Category')
```

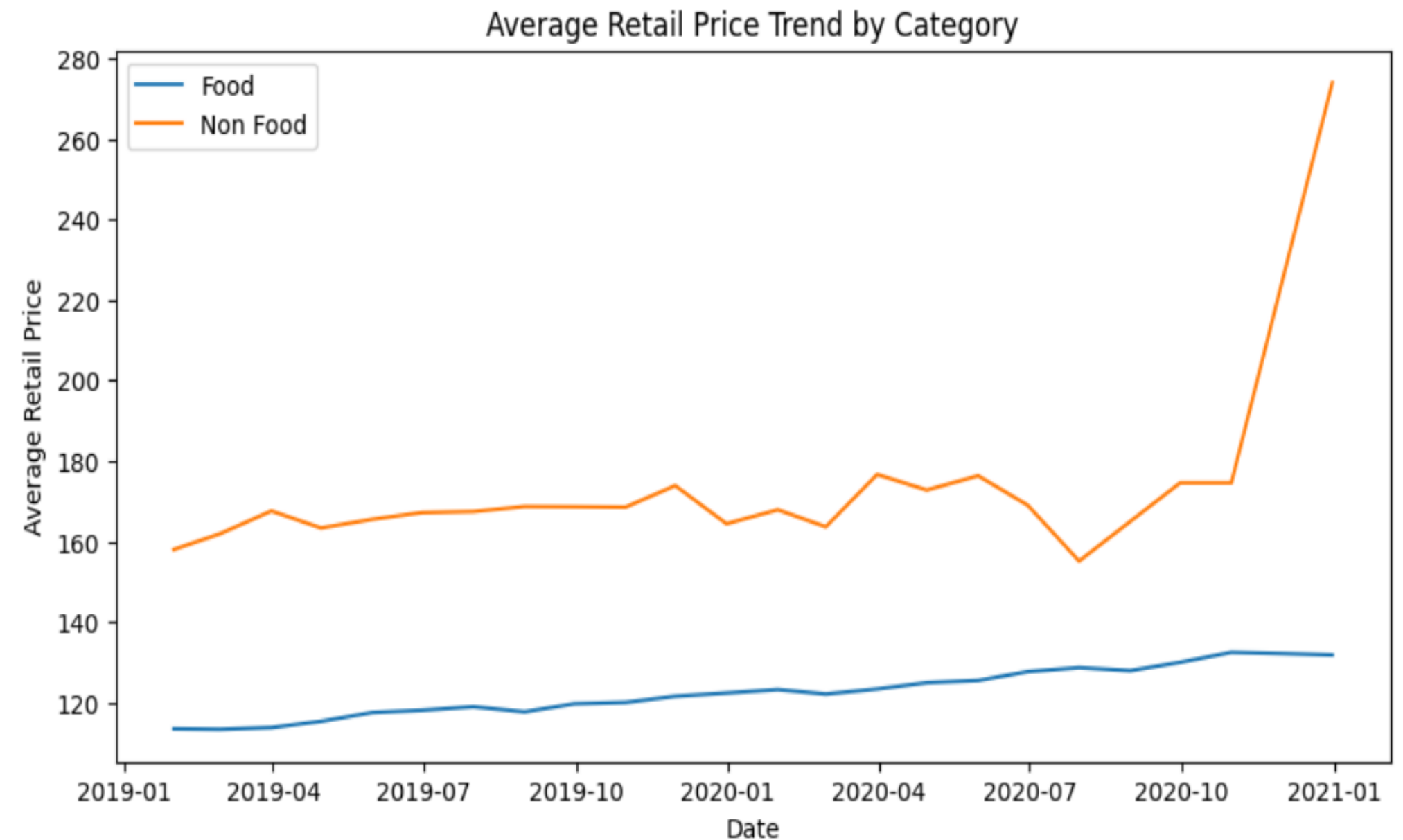
```
plt.xlabel('Date')
```

```
plt.ylabel('Average Retail Price')
```

```
plt.legend()
```

```
plt.show()
```

|   | Date       | Category | Retail_Price |
|---|------------|----------|--------------|
| 0 | 2019-01-31 | Food     | 113.435273   |
| 1 | 2019-01-31 | Non Food | 157.977395   |
| 2 | 2019-02-28 | Food     | 113.315042   |
| 3 | 2019-02-28 | Non Food | 161.892308   |
| 4 | 2019-03-31 | Food     | 113.749149   |



Business Insight:

This analysis reveals how commodity prices change over time across categories.

It helps identify trends, seasonal fluctuations, and supports forecasting and procurement planning.



## Key Insights

- Commodity prices are highly unevenly distributed, with a small number of premium or industrial commodities having extremely high prices compared to most products.
- A few major centres contribute a significant portion of commodity market data, making them key monitoring hubs.
- Commodity data reporting dropped significantly during the COVID-19 lockdown period.
- Non-food commodities generally have higher average prices than food commodities.
- Food commodities occasionally experience sharp price spikes despite lower average prices.
- Northeastern states consistently show the highest average commodity prices due to logistical challenges.
- Non-food commodity prices are more volatile and sensitive to market disruptions.

## Business Recommendations

- Segment commodities into Low, Medium, and High-Price categories and develop separate pricing, forecasting, and inventory strategies for each segment.
- Use these centres as benchmark markets while expanding data collection in less represented regions.
- Adopt automated and digital data collection systems to ensure business continuity during future disruptions.
- Implement separate pricing and forecasting models for Food and Non-Food categories.
- Maintain safety stock and improve supply chain monitoring for essential food products.

## Conclusion

Based on the Univariate, Bivariate, and Multivariate analyses, commodity prices are influenced by product category, location, and market trends. Food items are generally stable, while non-food items are more expensive and volatile. Regional factors and events like COVID-19 significantly impact prices. Overall, data-driven strategies can improve pricing, supply chain efficiency, and business decision-making.

# THANK YOU

Presented by: Nupur

